Drew University - CSCI 260 - Fall 2024
In-Class Exercise 1 - Data Representation

Instruction: Please work in groups of 3 and write down your names for attendance. 09/11/2022
Names:

1. Given the following binary numbers, compute their decimal values when being treated as unsigned, sign magnitude, 1's complement, and 2's complement numbers. (Just give me the formula.)

   (a) $(0010010111)_2$

   Ans: Note that under either format, this is a positive number, so we always use the weighted-positional notation to compute its decimal value as $2^0+2^1+2^2+2^4+2^7 = 1+2+4+16+128 = (151)_{10}$.

   (b) $(1110000110)_2$

   Ans: Note that when considering the MSB as the sign, this is a negative number; so, we need to treat it differently under each of the format.

   In unsigned, this is $2^1+2^2+2^7+2^8+2^9 = (902)_{10}$.

   In signed magnitude, we have $-(2^1+2^2+2^7+2^8) = (-390)_{10}$ instead; i.e., the MSB is the negative sign, not something that contributes to the magnitude.

   In 1's complement, we first remember the negative sign, and then flip all the bits to obtain $(0001111001)_2$. Thus, we compute the number as $-(2^0+2^3+2^4+2^5+2^6) = (-121)_{10}$.

   In 2's complement, we again first remember the negative sign, and then flip all the bits and add 1 to that to obtain $NOT(1110000110)+1 = 0001111001+1 = (0001111010)_2$; hence, the decimal value of the number is $-(2^1+2^3+2^4+2^5+2^6) = (-122)_{10}$.

2. Given the following decimal numbers, compute their 12-bit 2's complement binary representations and convert the results into their corresponding hexadecimal forms.

   (a) $(-56)_{10}$ using short division.

   Ans: Note that in 2's complement, this is a negative number, so we need to first compute the binary representation for its magnitude, and use 2's complement to negate that magnitude.

   $2 \mid 56$ **0**
   $2 \mid 28$ **0**
   $2 \mid 14$ **0**
   $2 \mid 7$ **1**
   $2 \mid 3$ **1**
   1

   Now, by tracing the remainders backward, we have $(56)_{10} = (11000)_2$. Note that we will need to add and extend the sign, 0, to make it 12-bit long; i.e., $(000000111000)_2$.
   Finally, by 2's complement, $(-56)_{10} = NOT(000000111000)+1 = (111111001000)_2$. To convert it to hexadecmial, we look at each of the 4-bit segments; hence, $(1111\ 1100\ 1000)_2$ = 0xFC8.

(b) $(175)_{10}$ by subtracting powers of 2.

Ans: We iteratively find the largest 2's power that's less than or equal to the number we are converting; hence, $(175)_{10} = 128+47 = 128+(32+15) = 128+32+(8+7) = 128+32+8+(4+3) = 128+32+8+4+(2+1) = 2^0+2^1+2^2+2^3+2^5+2^7$, which is is weighted-positional notation. Hence, by adding and extending sign bits, $(175)_{10} = (000010101111)_2 = 0x0AF$.

3. Compute the following logical expressions.

(a) NOT(01010111 AND NOT(11010111))

Ans: = NOT(01010111 AND 00101000) = NOT(00000000) = 11111111

(b) 0011 OR (0110 AND 1101)

Ans: = 0011 OR 0100 = 0111

(c) (0011 OR 0110) AND (0011 OR 1101)

Ans: = 0111 AND 1111 = 0111

4. For 2's complement binary numbers, compute the followings and determine whether there is overflow in each of the results:

(a) 01100111 + 10110: Need to extend the sign for the second operand with 1s as it is negative; i.e., 11110110. We denote a carry as a dot on top of each column and note that the carry resulted from the MSB (a crry-out) is discarded.

	.	.	.		.	.		
	0	1	1	0	0	1	1	1
+	1	1	1	1	0	1	1	0
	0	1	0	1	1	1	0	1

Since we have a positive number add a negative number, it is hard to tell the parity of the sum, so we look at the carry-in versus the carry-out of the MSB. Since they are the same (both being 1), there's NO overflow.

To verify, this is $103 + (-10) = 93$ in decimal.

(b) 101111 + 101101: This addition gives us a sum of 011100 where a leading 1 is discarded. Since we have now two negative numbers added together yielding a positive sum, there is clearly an OVERFLOW. You can also check by either looking at the carrys or verify under decimal.

(c) 01100111 − 10110: To subtract, we are adding the negation of the second operand, by 2's complement, but before that, we extend its sign; i.e., $-10110 = -11110110 = NOT(11110110) + 1 = 00001010$. Now, we add the two operands.

				.	.	.		
	0	1	1	0	0	1	1	1
+	0	0	0	0	1	0	1	0
	0	1	1	1	0	0	0	1

Since we have positive minus negative (or equivalently, positive add positive) and the sum we get also being positive, there is NO overflow.

To verify, this is $103 - (-10) = 103 + 10 = 113$ in decimal; or we can see the carrys are both 0.